

Runbook: Managing NTFS ACLs on AD DS Azure file shares (mount + icaccls)

Applies to: SMB Azure file shares whose storage account uses **on-prem AD DS** identity (`directory_type = "AD"`).

Why this runbook (and not an API)

Azure Files can set Windows/NTFS ACLs via the FileREST API and the `RestSetAcls` PowerShell module — **but Microsoft supports that path only for Microsoft Entra Kerberos identity sources**. For **AD DS** accounts the only supported tools are `icaccls` and **Windows File Explorer**, both of which require mounting the share from a domain-joined host. So ACL changes here are an operational task, not a Terraform/API one.

If these accounts move to Entra Kerberos in future, switch to `RestSetAcls` for keyless, mount-free, CLI-friendly automation.

Prerequisites

- A **Windows host that is domain-joined** to the same AD DS domain the storage account is joined to, with **unimpeded network connectivity to a domain controller** and **TCP 445** open to `*.file.core.windows.net`.
 - GitHub Actions hosted runners are **not** domain-joined — use a **self-hosted runner or a jumpbox VM** on the domain for any automation.
- The share already has **share-level RBAC** assigned (this is the gatekeeper; NTFS ACLs refine it — the most restrictive of the two wins).
- Admin-level access to edit ACLs, via **one** of:
 - **Recommended (keyless):** the **Storage File Data SMB Admin** RBAC role on the account, then mount with your domain identity. Grants `takeOwnership` so you can `takeown` files whose ACL would otherwise deny you.
 - **Fallback (less secure):** the **storage account key** — immediate full access, no `takeown` needed, but it's a sensitive credential. Avoid unless identity-based mount is impossible.

Script

Parameterized wrapper around `net use + icaccls`. Mount with `net use` (not `New-PSDrive`) so `icaccls`/Explorer can see the drive.

```
#requires -Version 5.1
[CmdletBinding()]
param(
    [Parameter(Mandatory)] [string] $StorageAccountName,
    [Parameter(Mandatory)] [string] $ShareName,
    # Each grant: "IDENTITY:PERMISSION" in icaccls syntax, e.g.
    # "IPSOSGROUP\MDM-USR-GB-United_Kingdom-Users:(OI)(CI)(M)"
    # "IPSOSGROUP\az-uk-azfiles-admins:(OI)(CI)(F)"
    [Parameter(Mandatory)] [string[]] $Grants,
```

```

    [string] $DriveLetter = 'Z',
    # Optional: mount with account key instead of the SMB Admin role (less
    secure).
    [string] $StorageAccountKey
)

$errorActionPreference = 'Stop'
$unc = "\\$StorageAccountName.file.core.windows.net\$ShareName"
$drive = "${DriveLetter}:"

# Idempotency: drop any stale mapping on this drive/UNC first.
cmd /c "net use $drive /delete /y" 2>$null | Out-Null
cmd /c "net use $unc /delete /y" 2>$null | Out-Null

try {
    if ($StorageAccountKey) {
        Write-Warning "Mounting with the storage account key (less secure)."
        cmd /c "net use $drive $unc /user:localhost\$StorageAccountName
$StorageAccountKey" | Out-Null
    }
    else {
        # Keyless: relies on the caller holding 'Storage File Data SMB Admin' and
        a valid Kerberos ticket.
        cmd /c "net use $drive $unc" | Out-Null
    }

    foreach ($g in $Grants) {
        Write-Host "icacls grant: $g"
        # /grant (no ':r') ADDS to the existing ACL; use /grant:r to REPLACE that
        identity's entry.
        icacls $drive /grant "$g"
        if ($LASTEXITCODE -ne 0) {
            # If denied by an existing ACL and you hold SMB Admin, take ownership
            then retry.
            Write-Warning "Grant failed (exit $LASTEXITCODE); attempting takeown
            then retry."
            takeown /F $drive /R /D Y | Out-Null
            icacls $drive /grant "$g"
        }
    }
}
finally {
    cmd /c "net use $drive /delete /y" 2>$null | Out-Null
}

```

Run for one share

```

.\Set-AzFilesAcl.ps1 -StorageAccountName azemeaukfs02 -ShareName data -Grants @(
    'IPSOSGROUP\az-uk-azfiles-admins:(OI)(CI)(F)',           # admins: full,
    inherited
    'IPSOSGROUP\MDM-USR-GB-United_Kingdom-Users:(OI)(CI)(M)' # UK users: modify,

```

```
inherited
)
```

Run across many accounts

```
$targets = @(
    @{ Account = 'azemeaukfs02'; Share = 'data' },
    @{ Account = 'azemeadeufs01'; Share = 'data' }
    # ... one row per account/share
)
$grants = @(
    'IPSOSGROUP\az-uk-azfiles-admins:(OI)(CI)(F)',
    'IPSOSGROUP\MDM-USR-GB-United_Kingdom-Users:(OI)(CI)(M)'
)
foreach ($t in $targets) {
    .\Set-AzFilesAcl.ps1 -StorageAccountName $t.Account -ShareName $t.Share -
    Grants $grants
}
```

icacls quick reference

Rights	Meaning	Inheritance	Meaning
(F)	Full control	(OI)	Object inherit (files)
(M)	Modify (read/write/delete)	(CI)	Container inherit (subdirs)
(RX)	Read & execute	(IO)	Inherit only (not this object)
(R)	Read		
(W)	Write		

Typical directory grant: `DOMAIN\Group:(OI)(CI)(M)` → modify, inherited by all files and subfolders.

Caveats

- **BUILTIN\Administrators is empty** on Azure Files and cannot be populated — don't rely on it. Use your AD groups.
- **Most-restrictive wins:** effective access = min(share-level RBAC role, NTFS ACL). A user with SMB Share **Reader** can't write even if NTFS says Modify.
- **takeown needs the SMB Admin role** (takeOwnership data action). With only Elevated Contributor + an existing Full-Control ACL you can edit ACLs without it.
- **Don't** use `New-PSDrive` to mount — the share won't be visible to `icacls`/Explorer.
- Keep this **out of hosted CI**; run only from a domain-joined host/self-hosted runner.
- Verify with `icacls Z:\somefolder` after applying.